UNIVERSIDAD RAFAEL LANDÍVAR

FACULTAD DE INGENIERÍA

LICENCIATURA EN INGENIERIA EN INFORMATICA Y SISTEMAS

MANEJO E IMPLEMENTACIÓN DE ARCHIVOS

PROYECTO 2

"DOCUMENTACIÓN TÉCNICA"

INTEGRANTE:	**CARNET:**
DIEGO ANGEL ALBERTO BARRIOS AJQUILL	1557125

QUETZALTENANGO, 8 NOVIEMBRE DE 2026

CAMPUS SAN ALBERTO HURTADO, S.J.

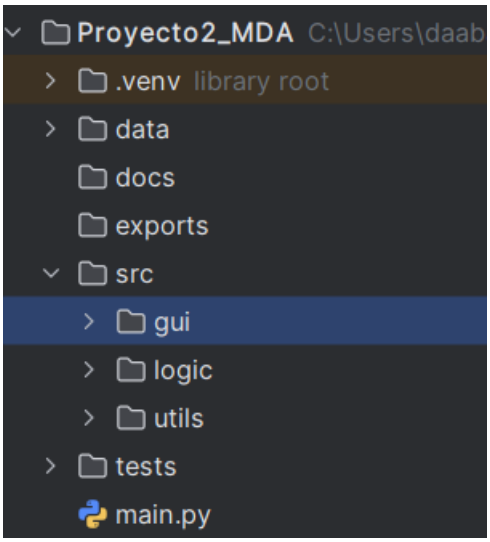
Estructura del Registro

Para cumplir con los requerimientos técnicos de acceso aleatorio, el archivo principal de almacenamiento (contactos.dat) fue diseñado utilizando registros de tamaño fijo. Todos los registros miden exactamente la misma cantidad de caracteres, rellenando los espacios sobrantes de cada campo con espacios en blanco (mediante la función ljust() de Python).

Campo	Tipo de Dato	Largo Asignado (Caracteres/Bytes)
ID	Entero (Único)	10
Nombre	Cadena de texto	40
Teléfono	Cadena de texto	15
Correo	Cadena de texto	40
Ciudad	Cadena de texto	44
Salto de línea (\n)	Control	1
	Tamaño Total por Registro:	**150 bytes**

Estructura organizada del proyecto

Se aplico una organización profesional del proyecto, dividida en módulos:



- **Src:** Donde se encuentra la lógica interna del proyecto, el cual está dividido en submódulos para separar responsabilidades.
    - **Gui:** Interfaz gráfica.
    - **logic:** Contiene las operaciones fundamentales para el manejo de los registros.
    - **Utils:** Tareas secundarias que exportan datos o leen metadatos.

- **Data:** Archivos que el programa lee y escribe constantemente para funcionar.
- **Exports:** Es la carpeta donde se almacena todo lo que se crea desde el menú, manteniendo la raíz del proyecto limpia.
- **Tests:** Realiza pruebas matemáticas para certificar que los registros miden exactamente los caracteres establecidos y que el salto con seek() llega al lugar correcto.
- **Docs:** Documentación del proyecto
- **main.py:** Es el punto de entrada de la aplicación, el cual tiene como única tarea arrancar el sistema e invocar a la ventana principal.

Implementación del Acceso Aleatorio

El acceso aleatorio permite ubicar el puntero de lectura/escritura directamente en la posición donde se encuentra un registro específico, sin necesidad de recorrer el archivo secuencialmente desde el inicio, esto se logra mediante la función seek() de Python y cálculos matemáticos simples basados en el tamaño fijo del registro (150 bytes).

Uso de seek() y tell()

Al agregar un nuevo contacto, se calcula su posición utilizando el número de registro asignado. Además, se utiliza tell() para confirmar y mostrar en consola la posición actual del puntero antes de escribir los datos en el archivo:

```
def agregar_contacto(ruta, numero_registro, id_val, nombre, telefono,
correo, ciudad):
    registro = formatear_registro(id_val, nombre, telefono, correo,
ciudad)

    with open(ruta, "r+", encoding="utf-8") as archivo:
        posicion = numero_registro * TAMANO_REGISTRO
        archivo.seek(posicion)

        print(f"Escribiendo registro en la posición del archivo (bytes):
{archivo.tell()}")
        archivo.write(registro)
```

Modificación
Para la modificación de un contacto existente se utilizó el acceso aleatorio es fundamental, ya que permite sobrescribir únicamente los 150 bytes del registro específico utilizando el modo de apertura "r+", sin necesidad de crear archivos temporales o reescribir todo el archivo de contactos.

Funcionamiento del Índice Primario

El índice primario actúa como un diccionario de búsqueda rápida. Se almacena en un archivo independiente (indice.json) y se carga en memoria RAM al iniciar la aplicación. Este índice relaciona directamente la clave principal (el ID del contacto) con su respectivo número de registro dentro del archivo físico.

Eficiencia Índice vs. búsqueda Secuencial:

Si se utilizara una búsqueda secuencial, el sistema tendría que abrir el archivo y leer línea por línea, comparando el ID buscado con el de cada registro, donde en un archivo con miles de contactos, lo que implicaría un alto costo computacional y de operaciones de entrada/salida. En cambio, con el índice primario en memoria, la búsqueda del ID es inmediata ya que una vez obtenido el número de registro desde el índice, el sistema calcula la posición exacta (num_registro × 150) y utiliza seek() para acceder a los datos de forma directa en disco lo cual reduce el tiempo de búsqueda a una complejidad de tiempo constante, haciendo el sistema drásticamente más eficiente.

Verificación de Integridad (SHA-256)

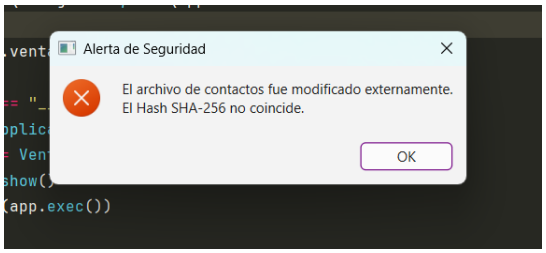
El sistema protege los datos calculando y comparando un hash criptográfico SHA-256 del archivo contactos.dat si el archivo es alterado manualmente (por ejemplo, editado desde el Bloc de notas), el hash actual no coincidirá con el almacenado en contactos.sha256, disparando una alerta de seguridad.

Original:

```
1234      Manuela                                12345678       mman@gmail.com
Toto
```

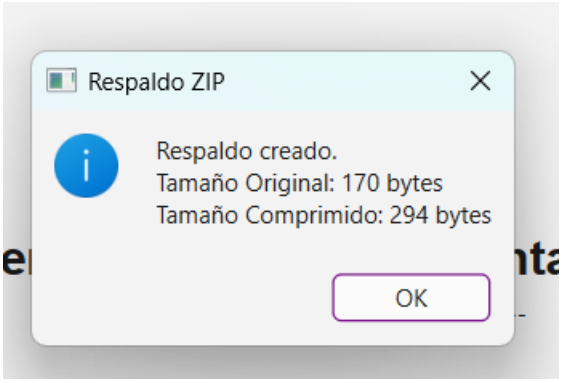
Editado:

```
1234      Manuela                                125678       mman@gmail.com
Toto
```



Respaldo ZIP y Comparación de Tamaños

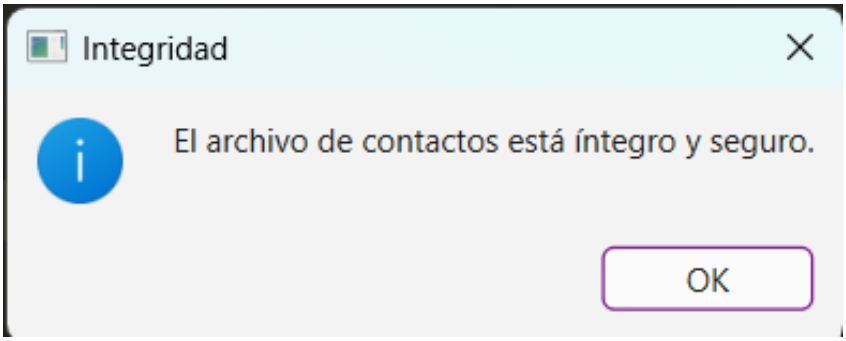
La opción de herramientas genera un archivo comprimido que incluye los archivos críticos del sistema desde el apartado de herramientas.



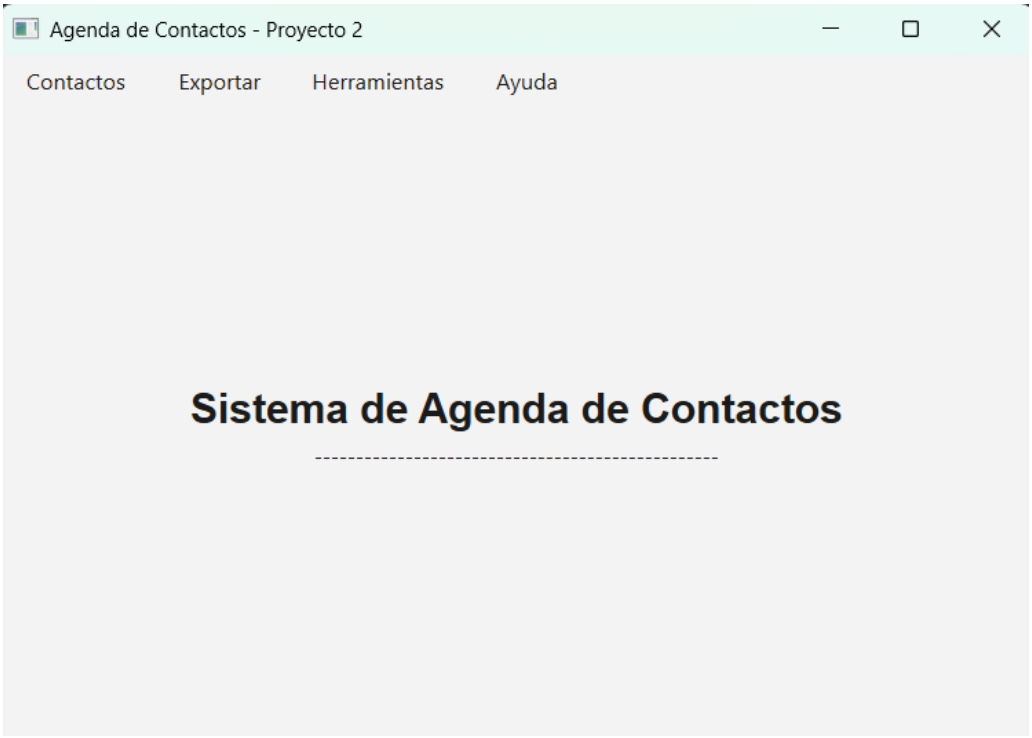
Capturas de Funcionamiento General

Evidencias del funcionamiento del menú principal, la interfaz gráfica y las opciones implementadas.

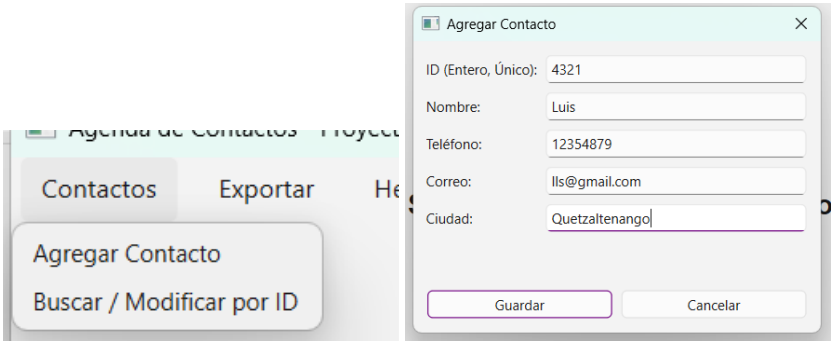
Verificación de integridad inicial:



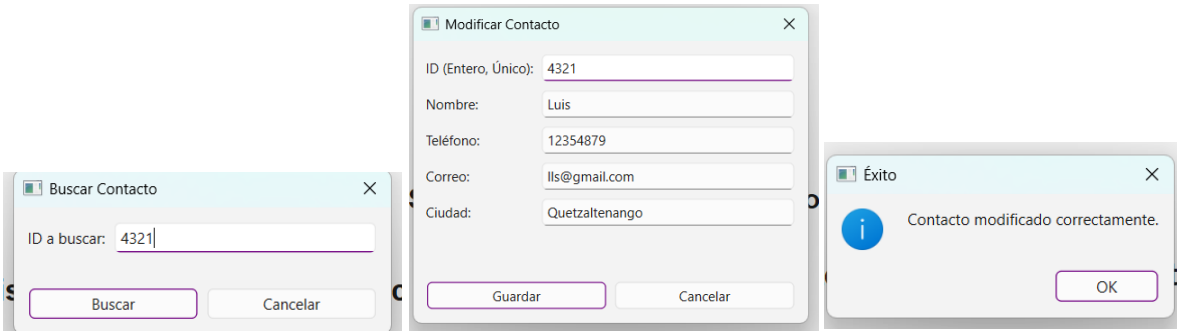
Menú principal:



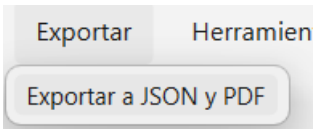
Agregar un contacto:

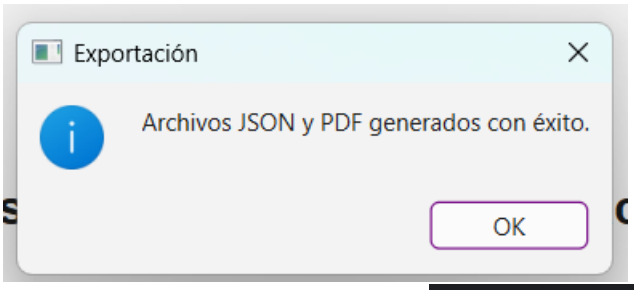


Buscar y modificar un contacto:

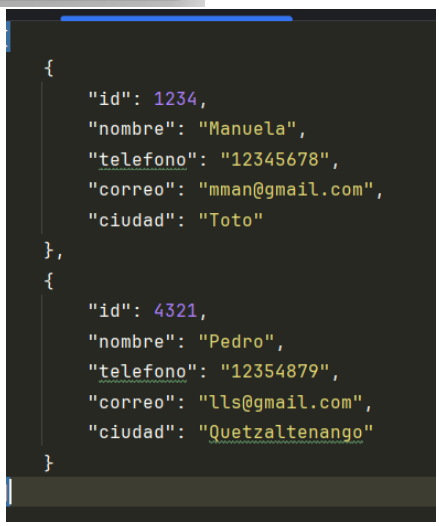


Exportar:

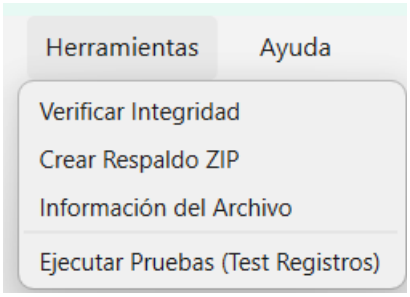




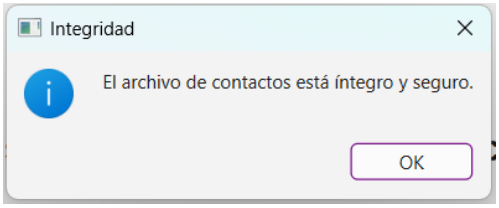




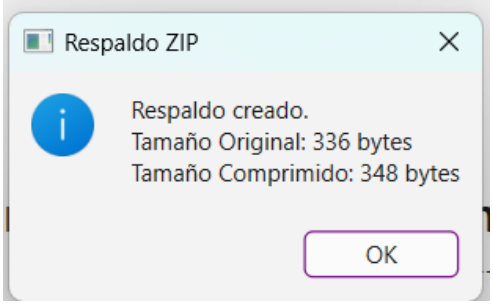
Herramientas:

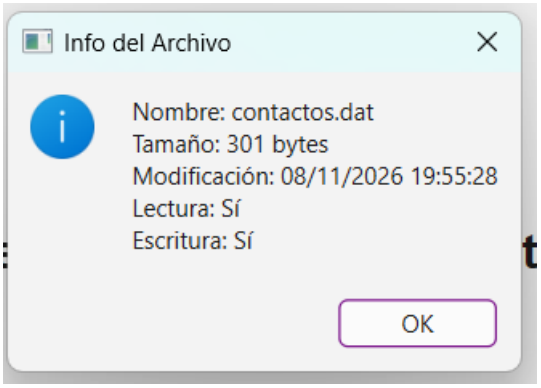


Re-calculación de hash:



Creación y comapación del zip:



Metadatos del archivo: 

Verificación del seek: 